

MODULE – 1

Approaches to Software Design

- Software design is a process to conceptualize the software requirements into software implementation.
- Software design takes the user requirements as challenges and tries to find optimum solution.
- While the software is being conceptualized, a plan is chalked out to find the best possible design for implementing the intended solution.

There are multiple variants of software design:

1. Function Oriented Design

- In function-oriented design, the system is comprised of many smaller sub-systems known as functions.
- These functions can perform significant task in the system.
- The system is considered as top view of all functions.
- Function oriented design inherits some properties of structured design where divide and conquer methodology is used.
- This design mechanism divides the whole system into smaller functions, which provides means of abstraction by concealing the information and their operation.
- These functional modules can share information among themselves by means of information passing and using information available globally.
- Another characteristic of functions is that when a program calls a function, the function changes the state of the program, which sometimes is not acceptable by other modules.
- Function oriented design works well where the system state does not matter and program/functions work on input rather than on a state.

2. Object Oriented Design

- Object oriented design works around the entities and their characteristics instead of functions involved in the software system.
- This design strategies focuses on entities and its characteristics.
- The whole concept of software solution revolves around the engaged entities.

Object Modeling Using Unified Modeling Language (UML)

Basic Object-Oriented concepts

The elements used in OOP are:.

- 1) **Objects:** Objects are the real time entities, which take/occupies memory place. They have the characteristic features such as identity and value. For example, Shiva is an object, because, he is real time entity. He has characteristic feature, name, color, weight, height & he occupies some space. But, Shiva does not exist without class human. Figure below shows the representation of shiva object in OOP.

Object: Shiva
DATA Name, colour, weight, height
FUNCTIONS Running, walking, speaking

- 2) **Classes**: A class is a collection of objects of similar type. Once a class is defined, any number of objects can be created which belong to that class. These are static (compile-time) definitions of a new type of a collection of data and associated operations (procedures or functions) from which runtime instances called objects can be created. In other way, a class is a blueprint that describes the nature of something. For example, the class 'dog' would consist of characters shared by all dogs, such as type and hair color (characteristics) and the ability to bark and sit (behaviors).
- 3) **Instance**: One can have an instance of a class or a particular object. The instance is the actual object created at runtime. In programmer, the Lassie object is an instance of the Dog class. The set of values of the attributes of a particular object is called its state. The object consists of state and the behaviour that is defined in the object's class.
- 4) **Inheritance**: In OOP, inheritance provides reusability, like, adding additional features to an existing class without modifying it. This is achieved by deriving a new class from the existing one. The new class will have combined features of both the classes. Inheritance allows the addition of additional features to an existing class without modifying it.
- 5) **Data Abstraction (Information Hiding)**: Abstraction refers to the act of representing essential features without including the background details or explanations. An abstraction denotes the essential characteristics of an object that distinguish it from all other kinds of objects. The goal of abstraction is to isolate those aspects that are important for some purpose and suppress those aspects that are unimportant.
- 6) **Encapsulation**: Storing data and functions in a single unit (class) is encapsulation. Encapsulation is also known as information hiding. This is complementary to abstraction and is used to describe the process of defining the code and data, which form an object. Encapsulation conceals the functional details of the class from the objects that send messages to it.
- 7) **Polymorphism**: is the ability to take more than one form. An operation may exhibit different behaviours in different instances. The behaviour depends on the data types used in the operation. Polymorphism is extensively used in implementing inheritance. Polymorphism allows the programmer to treat derived class member, like their parent class' members.
- 8) **Message Passing**: Message passing is a process by which an object sends data to another object or asks the other object to invoke a method. Message Passing is nothing but sending and of information by the objects same as people exchange information. So this helps in building systems that simulate real life. Following are the basic steps in message passing:
- Creating classes that define objects and its behaviour.

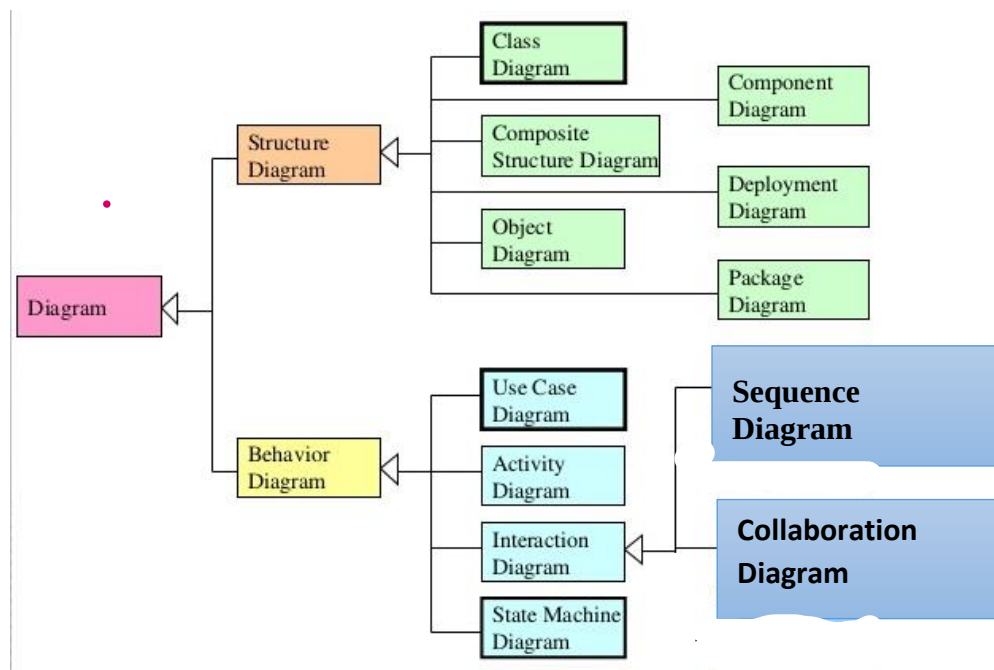
- ii. Creating objects from class definitions
- iii. Establishing communication among objects

UML

- UML (Unified Modeling Language) is a general-purpose, graphical modeling language in the field of Software Engineering
- UML is used to specify, visualize, construct, and document the artifacts (major elements) of the software system.
- UML is a visual language for developing software blueprints (designs).
 - A blueprint or design represents the model.
 - For example, while constructing buildings, a designer or architect develops the building blueprints. Similarly, we can also develop blueprints for a software system.
- UML is not a programming language; it is rather a visual language.
- The UML has the following features:
 - It is a generalized modeling language.
 - It is distinct from other programming languages like C++, Python, etc.
 - It is interrelated to object-oriented analysis and design.
 - It is used to visualize the workflow of the system.
 - It is a pictorial language, used to generate powerful modeling artifacts

Types of UML diagrams

- UML is linked with object-oriented design and analysis
- Diagrams in UML can be broadly classified as:
 1. **Structure Diagrams:** Capture static aspects or structure of a system
 2. **Behavior Diagrams:** Capture dynamic aspects or behavior of the system



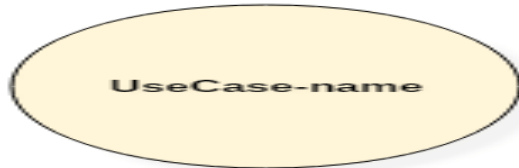
Uses Case Diagrams

- The purpose of a use case diagram in UML is to demonstrate the different ways that a user might interact with a system.
- It captures the dynamic behavior of a live system.

Use case diagram symbols and notation

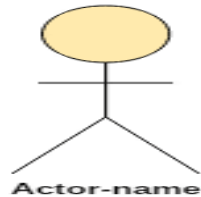
1. Use cases

- Horizontally shaped ovals that represent the different uses that a user might have.
- A use case represents a distinct functionality of a system, a component, a package, or a class.



2. Actors

- Stick figures that represent the people actually employing the use cases.
- A user is the best example of an actor
- One actor can be associated with multiple use cases in the system



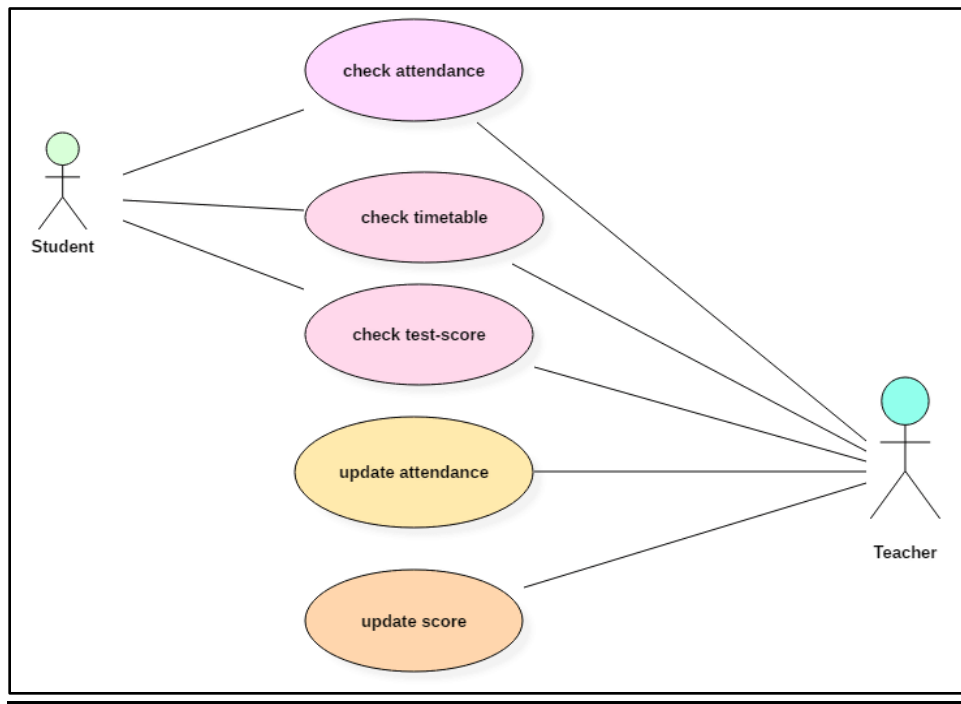
3. Associations (Communication Link)

- The participation of an actor in a use case is shown by connecting an actor to a use case by a solid link.
- Actors may be connected to use cases by associations, indicating that the actor and the use case communicate with one another using messages.

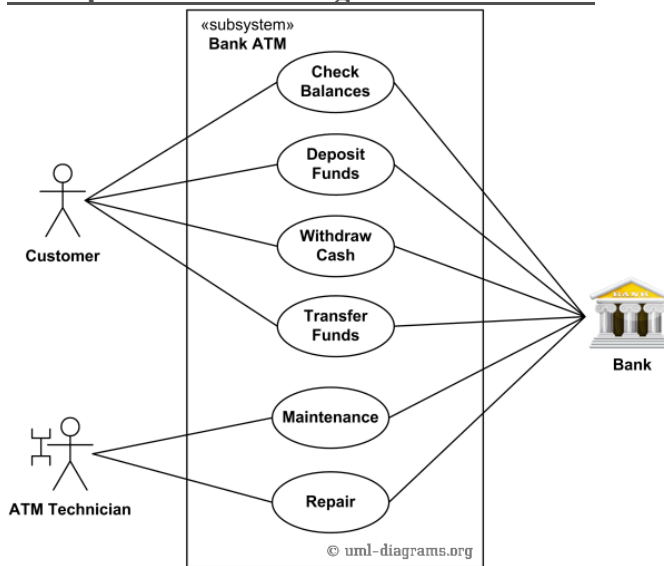
4. System boundary boxes

- The system boundary is potentially the entire system as defined in the requirements document.
- For large and complex systems, each module may be the system boundary.
- The system boundary is shown as a rectangle spanning all the use cases in the system.
- All use cases outside the box would be considered outside the scope of that system.

Example : Use Case Diagram – Student Management System



Example : Use Case Diagram – Bank ATM



CLASS DIAGRAM

- Class diagrams describes the attributes and operations of a class and also constraints imposed on the system.
- The most widely use UML diagram is the class diagram. It is the building block of all object-oriented software systems.
- Using class diagrams we can create the static structure of a system by showing system's classes, their methods and attributes.
- Class diagrams also help us identify relationship between different classes or objects.
- There are several software available which can be used online and offline to draw these diagrams Like Edraw max, lucid chart etc.

Class Notation

A class notation consists of three parts:

- **Class Name:**
 - The name of the class appears in the first partition.
- **Class Attributes:**
 - Attributes are shown in the second partition.
 - The attribute type is shown after the colon.
 - Attributes map onto member variables (data members) in code.
- **Class Operations (Methods):**
 - Operations are shown in the third partition.
 - They are services the class provides.
 - The return type of a method is shown after the colon at the end of the method signature.
 - The return type of method parameters are shown after the colon following the parameter name. Operations map onto class methods in code

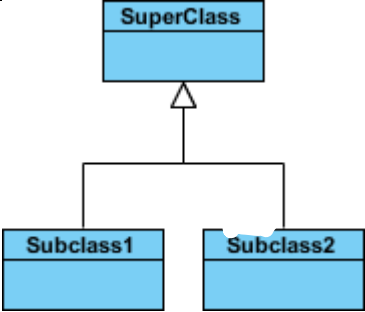
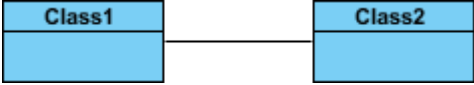
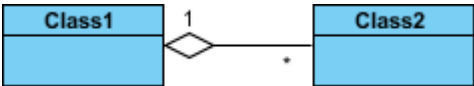
- The +, - and # symbols before an attribute and operation name in a class denote the visibility of the attribute and operation.
 - + denotes public attributes or operations
 - - denotes private attributes or operations
 - # denotes protected attributes or operations

MyClass
+attribute1 : int -attribute2 : float #attribute3 : int
+op1(in p1 : bool, in p2) : String -op2(input p3 : int) : float #op3(out p6) : Class6*

The graphical representation of the class – MyClass:

- MyClass has 3 attributes and 3 operations
- Parameter p3 of op2 is of type int
- op2 returns a float
- op3 returns a pointer (denoted by a *) to Class6

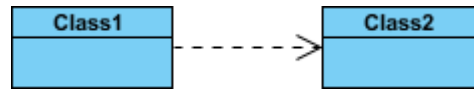
Relationships between classes

Inheritance (or Generalization): • Represents an "is-a" relationship. • An abstract class name is shown in italics. • SubClass1 and SubClass2 are specializations of Super Class. A solid line with a hollow arrowhead that point from the child to the parent class	 <pre>graph BT; Subclass1 --> SuperClass; Subclass2 --> SuperClass;</pre>
Simple Association: • A structural link between two peer classes. • There is an association between Class1 and Class2 A solid line connecting two classes	 <pre>graph LR; Class1 --- Class2;</pre>
Aggregation: A special type of association. It represents a "part of" relationship. • Class2 is part of Class1. • Many instances (denoted by the *) of Class2 can be associated with Class1. • Objects of Class1 and Class2 have separate lifetimes. A solid line with an unfilled diamond at the association end connected to the class of composite	 <pre>graph LR; Class1 "1" o-- "*" Class2;</pre>
Composition: A special type of aggregation where parts are destroyed when the whole is destroyed. • Objects of Class2 live and die with Class1. • Class2 cannot stand by itself. A solid line with a filled diamond at the association connected to the class of composite	 <pre>graph LR; Class1 "1" *-- "*" Class2;</pre>

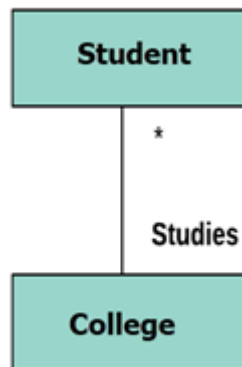
Dependency:

- Exists between two classes if the changes to the definition of one may cause changes to the other (but not the other way around).
- Class1 depends on Class2

A dashed line with an open arrow

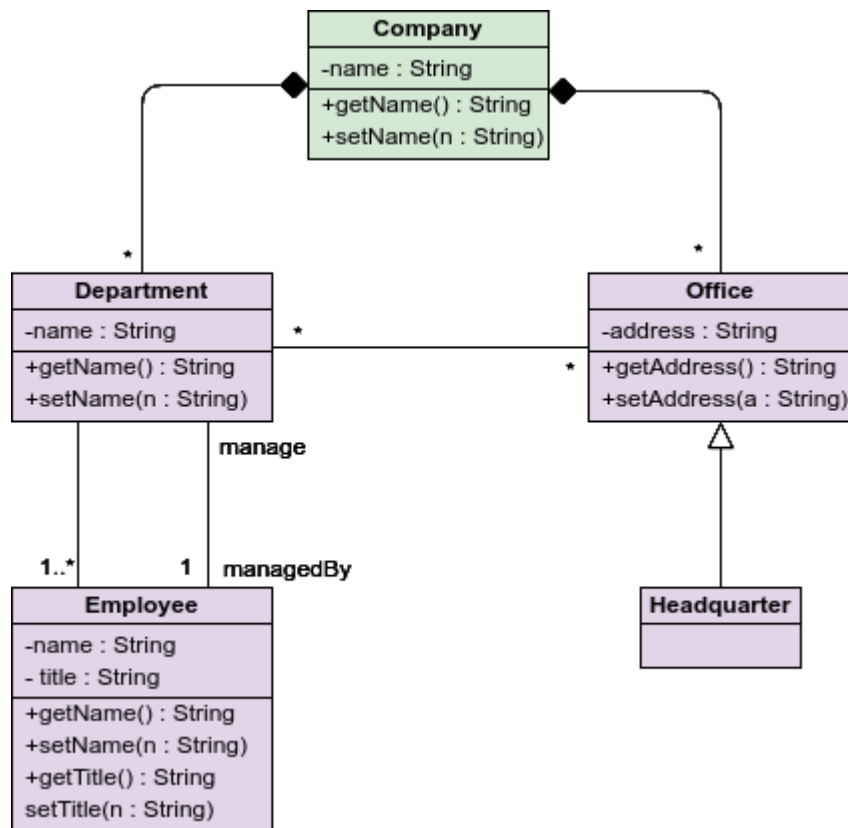
**Multiplicity**

- It means, how many objects of each class take part in the relationships
 - Exactly one - **1**
 - Zero or one - **0..1**
 - Many - **0..*** or *****
 - One or more - **1..***
 - Exact Number - e.g. 3..4 or 6
 - Or a complex relationship - e.g. 0..1, 3..4, 6.* would mean any number of objects other than 2 or 5



There are 100 students in one college. The college can have multiple students

Class Diagram : Company structure



Interaction Diagram

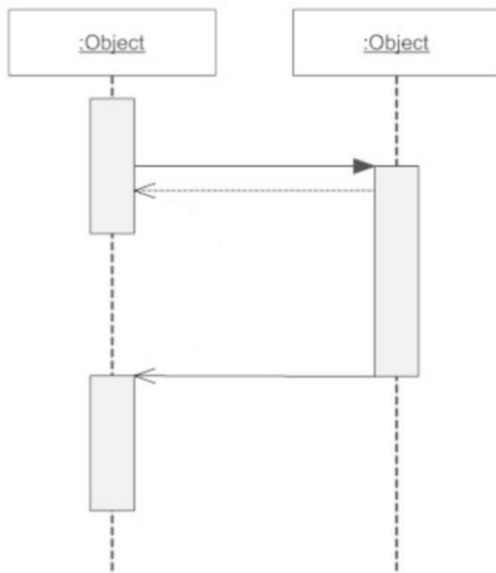
- used to describe some type of interactions among the different elements in the model. This interaction is a part of dynamic behavior of the system.
- This interactive behavior is represented in UML by two diagrams:
 - Sequence diagram** emphasizes on time sequence of messages and
 - Collaboration diagram** emphasizes on the structural organization of the objects that send and receive messages.

I. Sequence Diagram

- A Sequence Diagram simply depicts interaction between objects in a sequential order.
- The purpose of a sequence diagram in UML is to visualize the sequence of a message flow in the system
- Messages – Communication between objects is depicted using messages.
- The messages appear in a sequential order on the lifeline.

Basic Sequence Diagram Notations

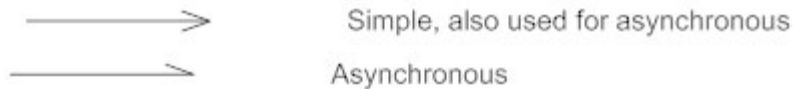
- Class Roles or Participants:** Class roles describe the way an object will behave in context.
- Activation or Execution Occurrence:** Activation boxes represent the time an object needs to complete a task.
- Messages:** Messages are arrows that represent communication between objects.
- Lifelines:** Lifelines are vertical dashed lines that indicate the object's presence over time.



Types of Messages in Sequence Diagrams

1. **Synchronous Message:** A synchronous message requires a response before the interaction can continue.  Synchronous

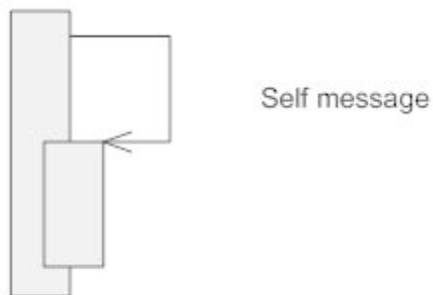
2. **Asynchronous Message:** Asynchronous messages don't need a reply for interaction to continue.



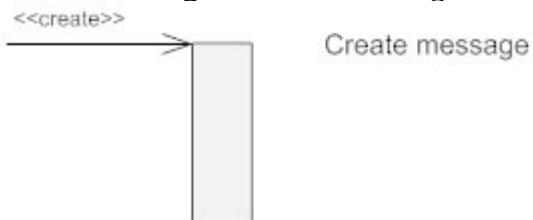
3. **Reply or Return Message:** A reply message is drawn with a dotted line and an open arrowhead pointing back to the original lifeline.



4. **Self-Message:** A message an object sends to itself, usually shown as a U-shaped arrow pointing back to itself.



5. **Create Message:** This is a message that creates a new object.



6. Delete Message

This is a message that destroys an object. It can be shown by an arrow with an x at the end.



7. Found Message

A message sent from an unknown recipient, shown by an arrow from an endpoint to a lifeline.

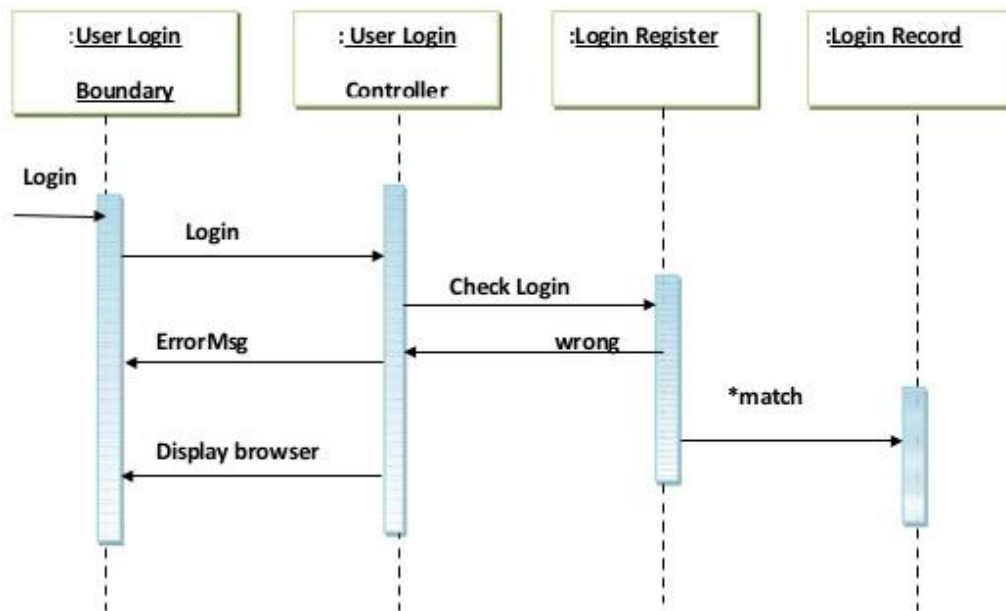


8. Lost Message

A message sent to an unknown recipient. It's shown by an arrow going from a lifeline to an endpoint, a filled circle or an x.



Sequence Diagram for User Login



II. Collaboration Diagram

- Collaboration represents the relationships and interactions among software objects.
- They are used to understand the object architecture within a system rather than the flow of a message as in a sequence diagram.

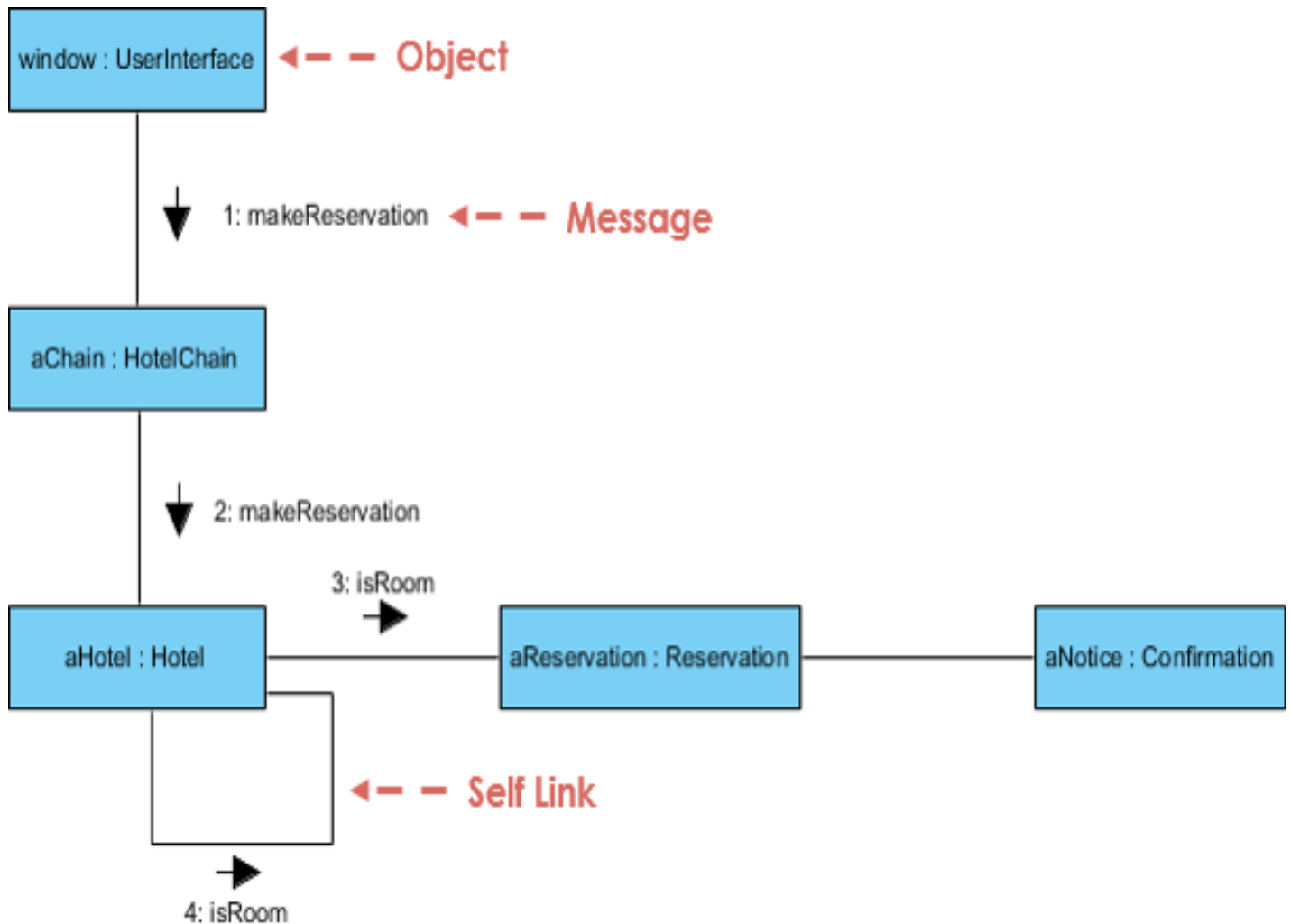
Collaboration Diagram Notations

1. **Objects**: An object is represented by an object symbol showing the name of the object and its class underlined, separated by a colon:

Object_name: class_name

2. **Links**: Links connect objects and actors and are instances of associations and each link corresponds to an association in the class diagram
3. **Messages**: A message is a communication between objects that conveys information with the expectation that activity will ensue. In collaboration diagrams, a message is shown as a labeled arrow placed near a link.

Collaboration Diagram Example : Hotel Reservation System



Activity Diagram

- Activity diagram is another important diagram in UML to describe the dynamic aspects of the system. Activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system.

Purpose of Activity Diagrams

The purpose of an activity diagram can be described as –

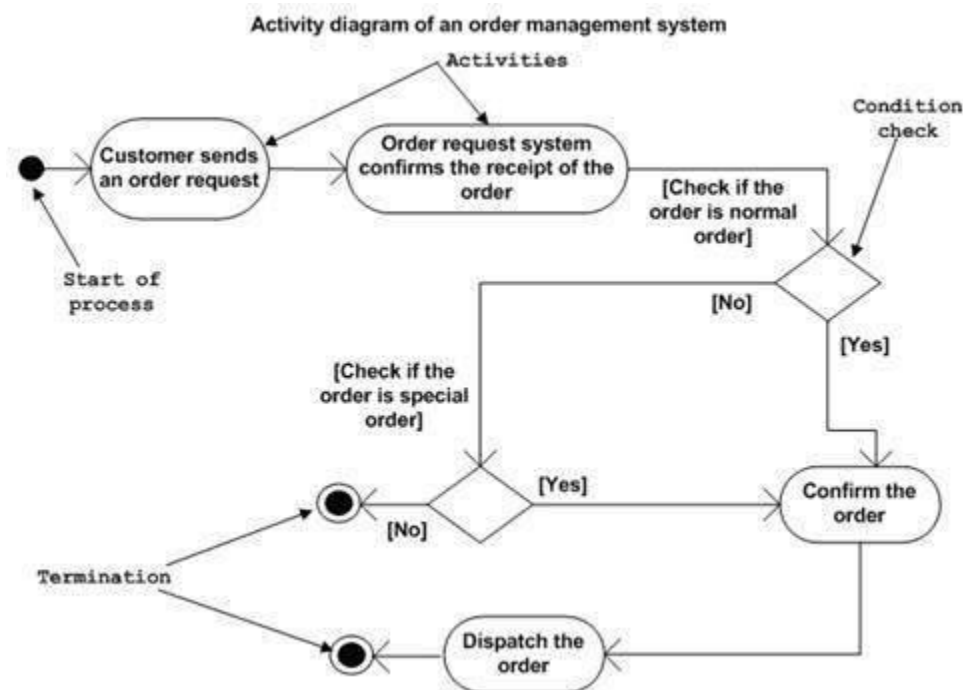
- Draw the activity flow of a system.
- Describe the sequence from one activity to another.
- Describe the parallel, branched and concurrent flow of the system.

Following is an example of an activity diagram for **order management system**. In the diagram, four activities are identified which are associated with conditions. One important point should be clearly understood that an activity diagram cannot be exactly matched with the code. The activity diagram is made to understand the flow of activities and is mainly used by the business users

Following diagram is drawn with the four main activities –

- Send order by the customer
- Receipt of the order
- Confirm the order
- Dispatch the order

After receiving the order request, condition checks are performed to check if it is normal or special order. After the type of order is identified, dispatch activity is performed and that is marked as the termination of the process.



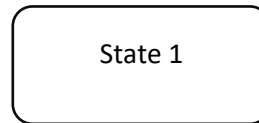
State chart Diagrams / State Machine Diagrams

State chart diagram is used to capture the dynamic aspect of a system.

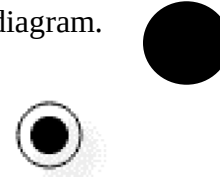
- It is also called a State machine or State Transition diagram
- An object goes through various states during its lifespan.
- The lifespan of an object remains until the program is terminated.
- The object goes from multiple states depending upon the event that occurs within the object (internal/External).
- Each state represents some unique information about the object.

Notations and symbol for state machine

➤ State box - a specific moment in the lifespan of an object



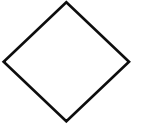
➤ Initial state – Used to indicate the beginning of a state machine diagram.



➤ Final state - used to indicate the end of a state machine diagram.



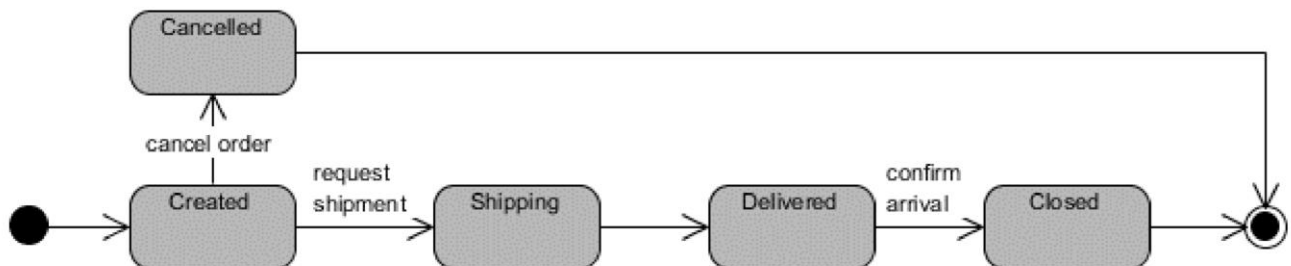
➤ Decision box - It contains a condition. Depending upon the result of an evaluated guard condition, a new path is taken for program execution.



➤ Transition - change in one state into another state which is occurred because of some event. A transition causes a change in the state of an object.



State chart diagram Example: Order Management System



Introduction to Java

What is Java?

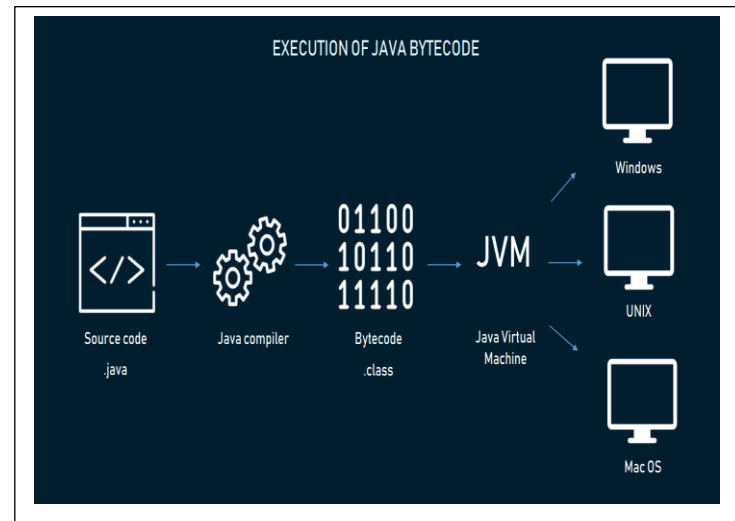
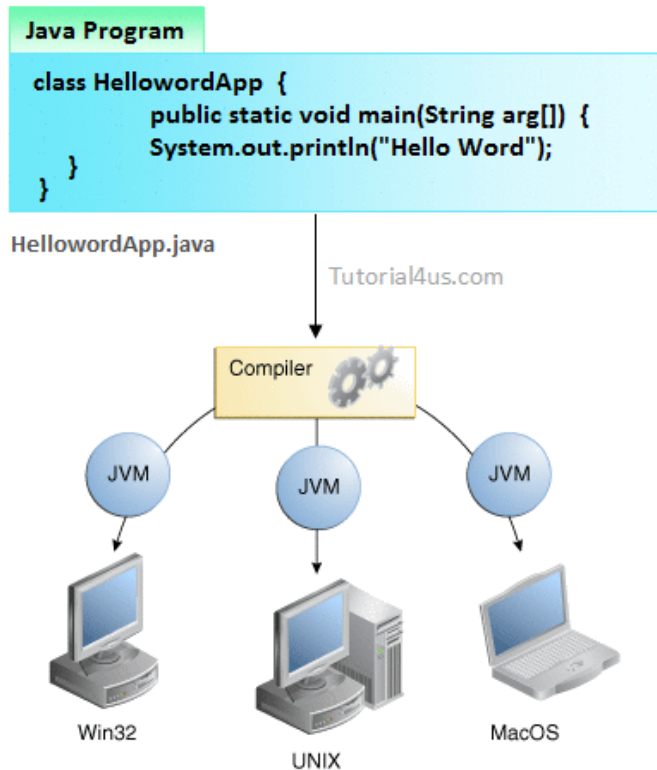
- Java is a **programming language** and a **platform**. Java is a high level, robust, object-oriented and secure programming language.
- Java was developed by *Sun Microsystems* (which is now the subsidiary of Oracle) in the year 1995. *James Gosling* is known as the father of Java. Before Java, its name was *Oak*. Since Oak was already a registered company, so James Gosling and his team changed the Oak name to Java.

Platform: Any hardware or software environment in which a program runs, is known as a platform. Since Java has a runtime environment (JRE) and API, it is called a platform.

How Java is Platform Independent?

A programming language or technology is said to be platform independent if and only if which can run on all available operating systems with respect to its development and compilation. (Platform represents Operating System). Java is a platform independent programming language, because when installing JDK software on your system then automatically JVM are

installed on your system. For every operating system separate JVM is available which is capable to read the **.class** file or **byte code**. When compile the Java code then .class file is generated by javac compiler these codes are readable by JVM and every operating system have its own JVM so JVM is platform dependent but due to JVM java language is become platform independent.



Java Programming Environment and the Java Runtime Environment (JRE)

What is the Java Programming Environment?

JAVA DEVELOPMENT KIT(JDK)

The Java Development Kit (JDK) is a software development environment used for developing Java applications and applets. It includes the Java Runtime Environment (JRE), an interpreter/loader (Java), a compiler (javac), an archiver (jar), a documentation generator (Javadoc) and other tools needed in Java development. Java is a recently developed, concurrent, class-based, object-oriented programming and runtime environment, consisting of:

- A programming language
- An API specification
- A virtual machine specification

JRE Components

JAVA RUNTIME ENVIRONMENT

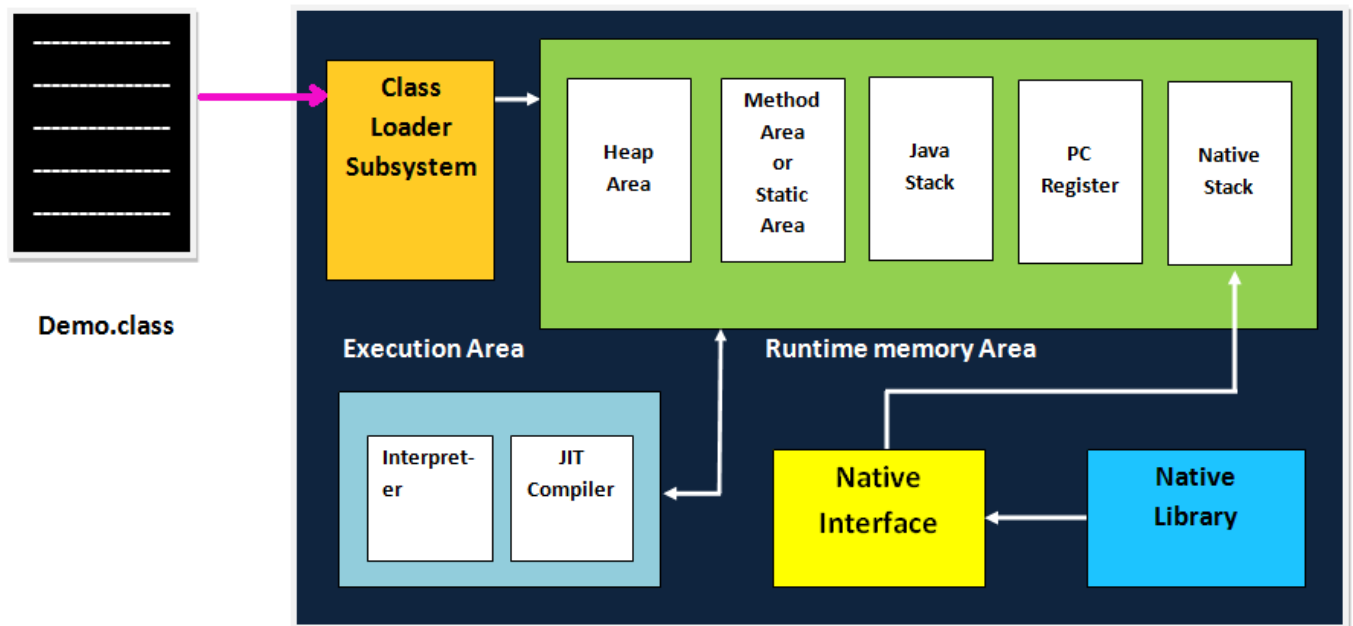
- JRE stands for “Java Runtime Environment” and may also be written as “Java RTE.” The Java Runtime Environment provides the minimum requirements for executing a Java application; it consists of the Java Virtual Machine (JVM), core classes, and supporting files.
- The JRE is the software environment in which programs compiled for a typical JVM implementation can run. The runtime system includes:
 - Code necessary to run Java programs, dynamically link native methods, manage memory, and handle exceptions
 - Implementation of the JVM

Development Platforms -Standard, Enterprise

- **Java Platform, Standard Edition (Java SE)**
 - Java Platform, Standard Edition (Java SE) is a computing platform for development and deployment of portable code for desktop and server environments. Java SE was formerly known as Java 2 Platform, Standard Edition (J2SE). The platform uses Java programming language and is part of the Java software-platform family.
- **Java Platform, Enterprise Edition (Java EE)**
 - The Java EE platform is built on top of the Java SE platform. The Java EE platform provides an API and runtime environment for developing and running large-scale, multi-tiered, scalable, reliable, and secure network applications.

JVM Architecture in Java

JVM (Java Virtual Machine) is a software. It is a specification that provides Runtime environment in which java bytecode can be executed.



Operation of JVM

JVM mainly performs following operations.

- Allocating sufficient memory space for the class properties.
- Provides runtime environment in which java bytecode can be executed
- Converting byte code instruction into machine level instruction.

JVM is separately available for every Operating System while installing java software so that **JVM is platform dependent**.

Note: Java is platform Independent but JVM is platform dependent because every Operating system have different-different JVM which is install along with JDK Software.

Class loader subsystem:

Class loader subsystem will load the **.class** file into java stack and later sufficient memory will be allocated for all the properties of the java program into following five memory locations.

- Heap area
- Method area
- Java stack
- PC register
- Native stack

Heap area:

In which object references will be stored.

Method area

In which static variables non-static and static method will be stored.

Java Stack

In which all the non-static variable of class will be stored and whose address referred by object reference.

PC Register

Which holds the address of next executable instruction that means that use the priority for the method in the execution process.

Native Stack

Native stack holds the instruction of native code (other than java code) native stack depends on native library. Native interface will access interface between native stack and native library.

Execution Engine

Which contains **Interpreter** and **JIT compiler** whenever any java program is executing at the first time interpreter will comes into picture and it converts one by one byte code instruction into machine level instruction JIT compiler (just in time compiler) will comes into picture from

the second time onward if the same java program is executing and it gives the machine level instruction to the process which are available in the buffer memory.

Note: The main aim of JIT compiler is to speed up the execution of java program.

Java Compiler

- Java compiler is a program that takes the text file work of a developer and compiles it into a platform-independent Java file.
- Java compilers include the Java Programming Language Compiler (javac), the GNU Compiler for Java (GCJ), the Eclipse Compiler for Java (ECJ) and Jikes.
- A **just-in-time (JIT) compiler** comes along with the Java VM. Its use is optional, and it is run on the platform-independent code. The JIT compiler then translates the code into the machine code for different hardware so that it is optimized for different architectures. Once the code has been (re-)compiled by the JIT compiler, it will usually run more quickly than the Java code that can only be executed one instruction at a time.

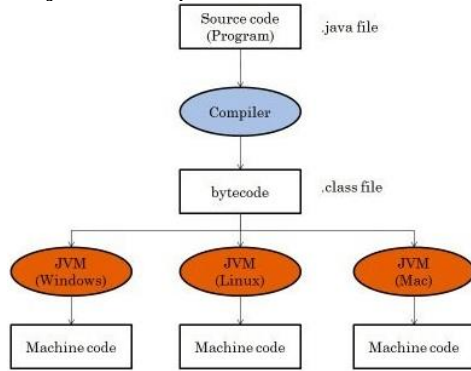
Java Bytecode

- Java bytecode is the instruction set for the Java Virtual Machine. It acts similar to an assembler which is an alias representation of a C++ code. As soon as a java program is compiled, java bytecode is generated. In more apt terms, java bytecode is the machine code in the form of a .class file. With the help of java bytecode we achieve platform independence in java.

How does it work?

- When write a program in Java, firstly, the compiler compiles that program and a bytecode is generated for that piece of code.
- After the first compilation, the bytecode generated is now run by the Java Virtual Machine and not the processor in consideration.
- Resources required to run the bytecode are made available by the Java Virtual Machine, which calls the processor to allocate the required resources. JVM's are stack-based so

they stack implementation to read the codes.



Advantage of Java Bytecode

- Platform independence is one of the soul reasons for which James Gosling started the formation of java and it is this implementation of bytecode which helps us to achieve this.
- Hence bytecode is a very important component of any java program.
- The set of instructions for the JVM may differ from system to system but all can interpret the bytecode.
- Bytecodes are non-runnable codes and rely on the availability of an interpreter to execute and thus the JVM comes into play.

Java Applet

- Applet is a special type of program that is embedded in the webpage to generate the dynamic content.
- It runs inside the browser and works at client side.

Advantage of Applet

There are many advantages of applet. They are as follows:

- It works at client side so less response time.
- Secured
- It can be executed by browsers running under many plateforms, including Linux, Windows, Mac Os etc.

Drawback of Applet

- Plugin is required at client browser to execute applet.

Java Buzzwords

The primary objective of Java programming language creation was to make it portable, simple and secure programming language. The features of Java are also known as **java buzzwords**.

1. Simple
 2. Object-Oriented
 3. Portable
 4. Platform independent
 5. Secured
 6. Robust
 7. Architecture neutral
 8. Interpreted
 9. High Performance
 10. Multithreaded
 11. Distributed
 12. Dynamic
1. **Simple**

- Java is very easy to learn, and its syntax is simple, clean and easy to understand. According to Sun, Java language is a simple programming language because:
 - Java syntax is based on C++ .
 - Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc.
 - There is no need to remove unreferenced objects because there is an Automatic Garbage Collection in Java.

2. Object-oriented

- Java is an object-oriented programming language.
- Everything in Java is an object.
- Object-oriented means we organize our software as a combination of different types of objects that incorporates both data and behavior.
- Object-oriented programming (OOPs) is a methodology that simplifies software development and maintenance by providing some rules.

3. Portable

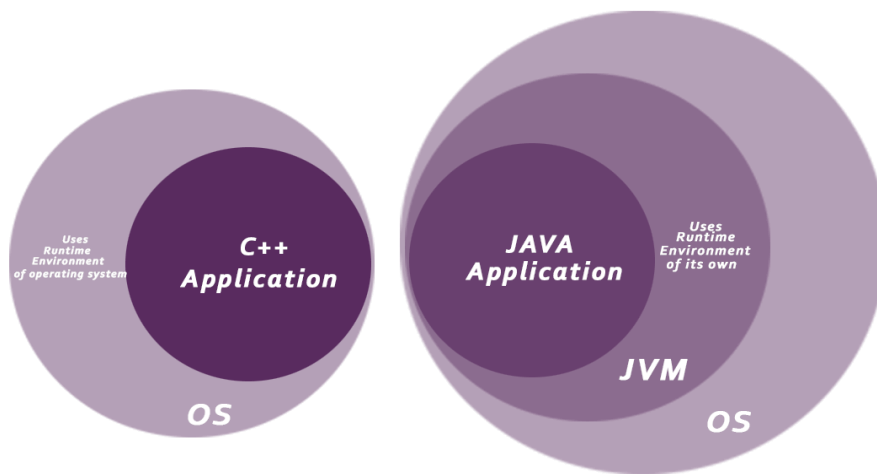
- Java is portable because it facilitates you to carry the Java bytecode to any platform. It doesn't require any implementation.

4. Platform Independent

- Java is platform independent because it is different from other languages like C, C++, etc. which are compiled into platform specific machines while Java is a write once, run anywhere language. A platform is the hardware or software environment in which a program runs. There are two types of platforms software-based and hardware-based. Java provides a software-based platform.
- Java code can be run on multiple platforms, for example, Windows, Linux, Sun Solaris, Mac/OS, etc. Java code is compiled by the compiler and converted into bytecode.
- This bytecode is a platform-independent code because it can be run on multiple platforms, i.e., Write Once and Run Anywhere (WORA).

5. Secured

- Java is best known for its security. With Java, we can develop virus-free systems. Java is secured because:
 - **No explicit pointer**
 - **Java Programs run inside a virtual machine sandbox**



- **Classloader:** Classloader in Java is a part of the Java Runtime Environment(JRE) which is used to load Java classes into the Java Virtual Machine dynamically. It adds security by separating the package for the classes of the local file system from those that are imported from network sources.
- **Bytecode Verifier:** It checks the code fragments for illegal code that can violate access right to objects.
- **Security Manager:** It determines what resources a class can access such as reading and writing to the local disk.

6. Robust

Robust simply means strong. Java is robust because:

- It uses strong memory management.
- There is a lack of pointers that avoids security problems.
- There is automatic garbage collection in java which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.
- There are exception handling and the type checking mechanism in Java. All these points make Java robust.

7. Architecture-neutral

- Java is architecture neutral because there are no implementation dependent features, for example, the size of primitive types is fixed. In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture. However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java.

8. Interpreted

- Java byte code is translated on the fly to native machine instructions and is not stored anywhere.
- The development process is more rapid and analytical since the linking is an incremental and light-weight process.

9. High-performance

- Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code. It is still a little bit slower than a compiled language (e.g., C++). Java is an interpreted language that is why it is slower than compiled languages, e.g., C, C++, etc.

10. Multi-threaded

- A thread is like a separate program, executing concurrently. We can write Java programs that deal with many tasks at once by defining multiple threads. The main advantage of multi-threading is that it doesn't occupy memory for each thread. It shares a common memory area. Threads are important for multi-media, Web applications, etc.

11. Distributed

- Java is distributed because it facilitates users to create distributed applications in Java. RMI and EJB are used for creating distributed applications. This feature of Java makes us able to access files by calling the methods from any machine on the internet.

12. Dynamic

- Java is a dynamic language. It supports dynamic loading of classes. It means classes are loaded on demand. It also supports functions from its native languages, i.e., C and C++.
- Java supports dynamic compilation and automatic memory management (garbage collection).

Java program structure

Documentation Section
Package Statement
Import Statement
Interface Statement
Class Definition
<pre> Main Method Class { //Main method definition }</pre>

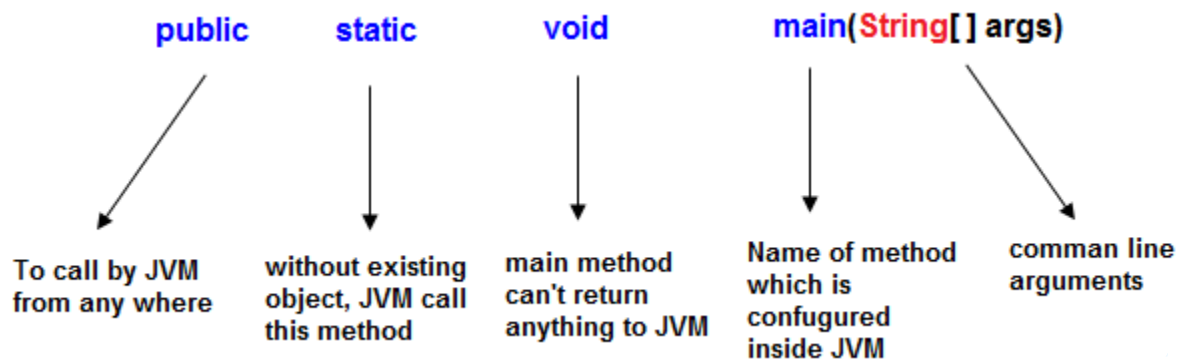
- **DOCUMENTATION SECTION:** It is a set of comment lines giving the name of the program, the author and the other details, which the programmer would like to refer.
- **PACKAGE STATEMENT:** The first statement allowed in Java files is a package statement. This statement declares a package name and informs the compiler that the classes defined here belong to this package.
 - Ex:package student;
 - It is optional because no need & our class to be part of packages.
- **IMPORT STATEMENT:** Similar to #include statement in C.
 - Ex:import student.test;

- This statement instructs the interpreter to load the test class contained in package student.
 - Using import statement , can access to classes that part of other named packages.
- **INTERFACE STATEMENT:** Interface like class but includes group of methods declaration.
 - Used when we want to implement multiple inheritance feature.
- **CLASS DEFINITION:**
 - Java program may contain multiple class definition.
 - Classes are primary feature of Java program.
 - The classes are used to map real world problems.
- **MAIN METHOD CLASS:**
 - Java stand alone program requires main method as starting point.
 - This is essential part of program
 - Main method creates object of various classes.
 - On reaching end of the main the program terminate and the control passes back to then operating system.
- Note:
 - Java Compiler will compile class that do not contain main() method.
 - If main() is not declared it give runtime error,because at compile time Compiler is not using main() but interpreter use at runtime.

package details	→	import java.io.*
class className	→	class Sum
{		
Data members;	→	int a, b, c;
user_defined method;	→	void display();
public static void main(String args[])		
{		
Block of Statements;	→	System.out.println("Hello Java !");
}		
}		

- A **package** is a collection of classes, interfaces and sub-packages. A sub package contains collection of classes, interfaces and sub-sub packages etc. java.lang.*; package is imported by default and this package is known as default package.
- **Class** is keyword used for developing user defined data type and every java program must start with a concept of class. "ClassName" represent a java valid variable name treated as a name of the class each and every class name in java is treated as user-defined data type.
- **Data member** represents either instance or static they will be selected based on the name of the class.

- **User-defined methods** represents either instance or static they are meant for performing the operations either once or each and every time.
- Each and every java program starts execution from the **main() method**. And hence main() method is known as program driver.
 - Since main() method must be accessed by every java programmer and hence whose access specifier must be **public**.
 - Since main() method of java executes only once throughout the java program execution and hence its nature must be **static**.
 - Since main() method of java is not returning any value and hence its return type must be **void**.
 - Each and every main() method of java must take array of objects of String.
 - String args[] is a String array used to hold command line arguments in the form of String values.
- **Block of statements** represents set of executable statements which are in term calling user-defined methods are containing business-logic.



Comments

- A comment is a non-executable statement that helps to read and understand a program especially when your programs get more complex. It is simply a message that exists only for the programmer and is ignored by the compiler. A good program should include comments that describe the purpose of the program, author name, date and time of program creation. This section is optional and comments may appear anywhere in the program.
- Java programming language supports three types of comments.
 - **Single line (or end-of line) comment:** It starts with a double slash symbol (//) and terminates at the end of the current line. The compiler ignores everything from // to the end of the line. For example:


```
// Calculate sum of two numbers
```
 - **Multiline Comment:** Java programmer can use C/C++ comment style that begins with delimiter /* and ends with */. All the text written between the delimiter is

ignored by the compiler. This style of comments can be used on part of a line, a whole line or more commonly to define multi-line comment. For example.

`/*calculate sum of two numbers */`

- **Documentation comments:** This comment style is new in Java. Such comments begin with delimiter `/**` and end with `*/`. The compiler also ignores this type of comments just like it ignores comments that use `/*` and `*/`. The main purpose of this type of comment is to automatically generate program documentation. The java doc tool reads these comments and uses them to prepare your program's documentation in HTML format. For example.

`/**The text enclosed here will be part of program documentation */`

Java Garbage Collection

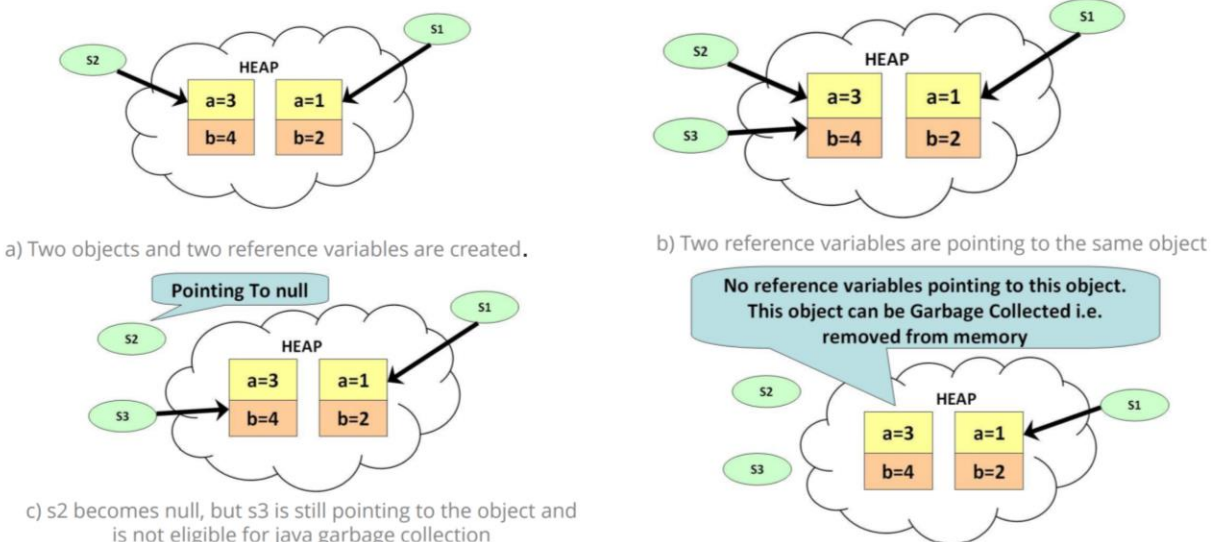
- In java, garbage means unreferenced objects. Garbage Collection is process of reclaiming the runtime unused memory automatically. In other words, it is a way to destroy the unused objects. In java it is performed automatically. So, java provides better memory management.

Advantage of Garbage Collection

- It makes java **memory efficient** because garbage collector removes the unreferenced objects from heap memory.
- It is **automatically done** by the garbage collector(a part of JVM) so we don't need to make extra efforts.

How can an object be unreferenced?

There are many ways:



Lexical Issues in Java

Java programs is a collection of White spaces , Identifiers , comments , Literals , Operators , Separators and Keywords.

-White Spaces

Java is a free form language. This means that you do not need to follow any special indentation rules. In java , white spaces is a space , tab or new line.

-Identifiers

Identifiers are used for class names , method names and variable names. An identifier may be any descriptive sequence of uppercase and lowercase letters , numbers or the underscore and dollar sign design.

-Literals

A constant value in java is created by using a literal representation of it. A literal can be used anywhere a value of its type is allowed.

-Comments

There are 3 types of comment in java. First is single line comment and the second one is multi line comment. The third type of comment is called documentation comment. It is used to produce an HTML file that documents your program. It begins with `/**` and ends with `*/`.

-Separators

There are few symbols in java that are used as separators. The most commonly used separator in java is the **semicolon** `;` . some other separators are **Parentheses** `()` , **Braces** `{}` , **Bracket** `[]` , **Comma** `,` , **Period** `.` .

- Java Keywords

There are 49 reserved keywords currently defined in java. These keywords cannot be used as names for a variable , class or method.

The **Keywords** are : abstract , assert , boolean , break , byte , case , catch , char , class , const , continue , default , do , double , else , extends , final , finally , float , for , goto , if , implements , import , instanceof , int interface , long , native , new , package , private , protected , public , return , short , static , strictfp , super , switch , synchronized , this , throw , throws , transient , try , void , volatile, while.

Java Class Libraries

The java environment relies on several built-in class libraries that contain many built-in methods that provide support for such things as I/O , string handling , networking and graphics.